

Chapter 1 Introduction

CHAPTER OBJECTIVES

- Introducing the Java programming language.
- Describe the steps involved in program compilation and execution.
- Explore the issues related to problem solving in general.
- Discuss the activities involved in the software development process.
- Present an overview of object-oriented principles.

This material is dedicated to the craft of building well-structured software. We start by taking a close look at a straightforward Java program and use it as a vehicle to introduce fundamental programming ideas. From there, we establish the conceptual foundation for larger-scale software construction, covering the essentials of problem-solving strategies, the lifecycle activities of software development and the core tenets of object-oriented programming.

1.1 The Java Programming Language

A computer is composed of two inseparable elements: hardware and software. The hardware components are the physical, tangible parts that carry out the actual computational work — chips, circuit boards, keyboards, monitors, cables, printers and all the other visible pieces you can touch. On their own, however, these components are incapable of doing anything meaningful without a set of instructions to guide them. A program is precisely that: an ordered sequence of instructions that the hardware carries out one by one. Programs are also commonly referred to as applications. Software, in turn, encompasses both the programs themselves and the data those programs operate on. It is the intangible counterpart to the physical machinery. When hardware and software work together, they form a powerful tool for tackling real-world problems.

KEY CONCEPT

A computer system consists of hardware and software that work in concert to help us solve problems.

Every program is authored in a specific programming language, which provides a defined vocabulary and set of symbols for expressing solutions to problems. A programming language establishes a precise collection of rules that governs how a programmer may combine its vocabulary and symbols into valid programming statements — the actual instructions that get carried out when the program runs.

Since the earliest days of computing, a vast number of programming languages have been invented. Throughout this book, we use the Java language to illustrate a wide range of

programming concepts and techniques. While our primary objective is to develop a deep understanding of core software development principles, a meaningful byproduct of that journey will be hands-on proficiency in writing Java programs.

Compared to many of its predecessors, Java is a relatively young programming language. It was conceived in the early 1990s by James Gosling at Sun Microsystems, officially unveiled to the public in 1995, and has since achieved remarkable and widespread adoption.

Java has continued to evolve since its original release. The contemporary form of the technology is broadly known as the Java 2 Platform, which is organized into three distinct editions:

- Java 2 Platform, Standard Edition (J2SE)
- Java 2 Platform, Enterprise Edition (J2EE)
- Java 2 Platform, Micro Edition (J2ME)

This book concentrates on the Standard Edition — the mainstream, general-purpose version of the language along with its accompanying tools. More specifically, our content is grounded in the latest release of that edition, J2SE 6.0.

Certain elements from earlier versions of Java have been deprecated, meaning they are regarded as outdated and discouraged from further use. Where it matters, we identify these deprecated features and present the recommended modern alternatives in their place.

KEY CONCEPT

This book focuses on the principles of object-oriented programming.

Java is fundamentally an object-oriented programming language. Objects serve as the primary building blocks from which programs are constructed. The principles underlying object-oriented software development are central to everything this book covers. We examine these concepts in depth later in this chapter and continue building on them throughout the entire text.

Accompanying the Java language itself is an extensive collection of pre-built software known as the Java standard class library. This library equips developers with ready-made capabilities for tasks such as rendering graphics, establishing network connections, and communicating with databases, among many other functions. The standard library is both vast and highly flexible. While a complete tour of every corner of it lies beyond the scope of this book, we will visit many of its most important and useful features.

Java enjoys widespread adoption in commercial settings across the globe and ranks among the most rapidly growing programming technologies ever developed. This means it is not only an excellent language for mastering programming fundamentals — it is also a highly practical skill that will continue to deliver real value throughout your career.

A Java Program

Let's take a look at a simple yet complete Java program. The program shown in Listing 1.1 outputs two lines of text to the screen — specifically, a well-known quotation attributed to Abraham Lincoln. The resulting output appears directly beneath the program listing.

All Java applications share a common underlying structure. Despite its brevity and straightforward purpose, this program packs in several key concepts worth examining carefully. Let's break it apart and take a closer look at each piece.

The opening lines of the program are comments, introduced by the `//` symbol and extending to the end of each line. Comments have no influence on what the program does; they exist solely to make the code more intelligible to human readers. Programmers are encouraged to insert comments throughout their code to clarify the program's intent and to describe any non-obvious logic. Any written material associated with a program — whether inline notes, user guides, or technical references — falls under the umbrella of documentation. Comments embedded directly within the source code are specifically referred to as inline documentation.

The remainder of the program constitutes a class definition. This particular class is named `Lincoln`, though virtually any valid name could have been chosen instead. The class definition spans from the opening brace (`{`) all the way to the closing brace (`}`) on the program's final line. Every Java program is built around one or more class definitions.

Nested within the class definition are additional comments that describe the purpose of the `main` method, which is defined immediately below them. A method is a named collection of programming statements grouped together to perform a specific task. Here, the method is called `main` and contains exactly two statements. Like a class definition, a method is also bounded by a pair of braces.

KEY CONCEPT

Comments do not affect a program's processing; instead, they serve to facilitate human comprehension.

```
// LISTING 1.1
//*****
// Lincoln.java
//
// Demonstrates the basic structure of a Java application.
//*****
public class Lincoln
{
    //-----
    // Prints a presidential quote.
    //-----
    public static void main(String[] args)
    {
        System.out.println("A quote by Abraham Lincoln:");

        System.out.println("Whatever you are, be a good one.");
    }
}
```

OUTPUT

A quote by Abraham Lincoln:
Whatever you are, be a good one.

Every Java application contains a `main` method, which serves as the program's entry point — the place where execution begins. Each statement within `main` is carried out sequentially, one after another, until the final statement is reached, at which point the program concludes and exits. The `main` method declaration is always preceded by the keywords `public`, `static`, and `void`, the significance of which will be addressed later in the text. The role of `String` and `args` does not come into play in this particular example; those will also be explained in due course.

The two statements inside `main` each invoke a method called `println` (short for "print line"). To invoke, or call, a method means to trigger its execution. The `println` method outputs the specified text to the screen. The text itself is represented as a string of characters enclosed in double quotation marks (`"`). When the program runs, it calls `println` to display the first line of text, then calls it again to display the second and since no further statements remain in `main`, the program terminates.

The code that actually runs when `println` is invoked is not defined anywhere in this program. The `println` method belongs to the `System.out` object, which is part of the Java standard class library. Strictly speaking, it is not a built-in feature of the Java language

itself, but it is universally available in any Java program. We will take a more thorough look at `println` in Chapter 2.

Comments

Let's explore comments in greater detail. Comments represent the only mechanism within a programming language that allows developers to express their thoughts and reasoning entirely independently of the executable code. Well-written comments offer genuine insight into the programmer's original intent and design decisions. Programs often remain in use for many years, accumulating modifications along the way. Even the original author may not recall the specifics of code written months or years earlier when revisiting it for updates. And more often than not, the person tasked with making those changes is someone entirely unfamiliar with the codebase. This makes thorough, meaningful documentation not a luxury, but a necessity.

As far as the Java compiler is concerned, the content of a comment can be any text at all. The compiler disregards comments entirely; they play no role whatsoever in how the program executes.

The comments in the `Lincoln` program illustrate one of two comment styles supported by Java:

```
// This is a comment.
```

This form begins with a double forward slash (`//`) and runs to the end of the current line. No characters may appear between the two slashes. Everything following the `//` on that line is treated as a comment and ignored by the compiler. A comment of this type can also be placed on the same line as code, serving as an inline annotation for that specific statement:

```
System.out.println ("Monthly Report"); // always use this title
```

The second comment style takes the following form:

```
/* This is another comment. */
```

Unlike the single-line variety, this style is not terminated by the end of a line. Instead, everything between the opening `/*` and the closing `*/` is treated as part of the comment — including any newline characters in between. This makes it well suited for comments that span multiple lines. No space is permitted between the slash and the asterisk in either the opening or closing delimiter.

When a second asterisk immediately follows the `/*` at the start of a comment, the resulting `/**` syntax enables the `javadoc` tool to automatically extract and generate external documentation from the comment's content.

These two comment forms can be combined in a variety of ways to produce different documentation styles:

```
// This is a comment on a single line.
```

```
//_____
```

```
// Some comments such as those above methods or classes
```

```
// deserve to be blocked off to focus special attention
```

```
// on a particular aspect of your code. Note that each of
```

```
// these lines is technically a separate comment.
```

```
//_____
```

```
/*
```

```
    This is one comment
```

```
    that spans several lines.
```

```
*/
```

KEY CONCEPT

Inline documentation should provide insight into your code. It should not be ambiguous or belabor the obvious.

Programmers frequently invest so much mental energy into writing code that documentation becomes an afterthought. Developing strong commenting habits early — and applying them consistently — is essential. Comments should be clearly written, often in complete sentences, and should add genuine value. They must neither state the obvious nor leave the reader guessing. The following examples illustrate what poor comments look like:

```
System.out.println ("hello"); // prints hello
```

```
System.out.println ("test"); // change this later
```

The first comment simply restates what the code already makes self-evident, contributing nothing of value. In fact, a comment that adds no meaning is worse than no comment at all — it clutters the code without aiding understanding. The second comment is vague to the point of being useless. What exactly needs to change? When should it change? And for what reason?

Identifiers and Reserved Words

The various names and terms that appear throughout a program are collectively called identifiers. In the `Lincoln` program, the identifiers present are: `class`, `Lincoln`, `public`, `static`, `void`, `main`, `String`, `args`, `System`, `out`, and `println`. These can be grouped into three distinct categories:

- names we ourselves chose while writing the program (`Lincoln` and `args`)
- names chosen by other programmers whose code we are making use of (`String`, `System`, `out`, `println`, and `main`)
- names reserved by the Java language for specific built-in purposes (`class`, `public`, `static`, and `void`)

When writing the program, we selected `Lincoln` as the class name, but countless other choices would have been equally valid — `Quote`, `Abe`, or `GoodOne`, for instance. The identifier `args` (an abbreviation for "arguments") is used here in a conventional way, though it could be replaced with almost any other valid identifier without affecting how the program works.

The identifiers `String`, `System`, `out`, and `println` were not invented by us — they were defined by the engineers who built the Java standard library. These names are not part of the Java language specification itself; they belong to a pre-written collection of reusable classes and methods that other programmers have made available for our use. We are simply leveraging what they built.

Reserved words are identifiers that carry a fixed, predefined meaning within the Java language and cannot be repurposed for anything else. They cannot be used as the name of a class, method, or variable. In the `Lincoln` program, the reserved words in use are `class`, `public`, `static`, and `void`.

Any identifier we define ourselves may consist of any combination of letters, digits, underscores (`_`), and dollar signs (`$`), with the sole restriction that it must not begin with a digit. Identifiers may be of any length. Consequently, names such as `total`, `label17`, `nextStockItem`, `NUM_BOXES`, and `$amount` are all perfectly valid, whereas `4th_word` and `coin#value` are not.

KEY CONCEPT

Java is case sensitive. The uppercase and lowercase versions of a letter are distinct.

Java treats uppercase and lowercase letters as entirely separate characters. This means two identifiers that are identical in every way except letter case are considered completely different names. For example, `total`, `Total`, `ToTaL`, and `TOTAL` are four distinct identifiers. As you might expect, relying on case differences alone to distinguish between identifiers is a recipe for confusion and should be avoided entirely.

Identifier

A Java Letter includes the 26 English alphabetic characters in both uppercase and lowercase, the `$` and `_` (underscore) characters, as well as alphabetic characters from other languages. A Java Digit includes the digits 0 through 9.

An identifier consists of a letter followed by zero or more letters and digits.

Examples: `total`, `MAX_HEIGHT`, `num1`, `computeWage`, `System`

KEY CONCEPT

Identifier names should be descriptive and readable.

Although the Java language places no restrictions on the style of identifiers beyond the basic formation rules, adopting a consistent naming convention for each category of identifier significantly improves readability and comprehension. Java has well-established conventions for this purpose, and we adhere to them throughout this book. For instance, class names are written in title case, where the first letter of each word is capitalized. The preferred style for each type of identifier is introduced when that identifier type first appears in the text.

While identifiers can be arbitrarily long, name selection deserves deliberate thought. Names should be descriptive without being wordy. Vague single-character names like `a` or `x` should generally be avoided — unless the brevity is itself meaningful, such as using `x` and `y` for coordinates on a two-dimensional grid. Equally, excessively verbose names like `theCurrentItemBeingProcessed` are burdensome; something like `currentItem` conveys the same meaning in far fewer characters. In practice, overly terse names are a far more common problem than overly long ones.

Readability should always be a priority. Abbreviations, in particular, need to be handled carefully. An identifier like `curStVal` might seem like an efficient shorthand for "current stock value" to the person who wrote it, but to anyone else — or even to the same person revisiting the code weeks later — its meaning may be far from clear.

In Java, a name is formed by stringing together a series of identifiers separated by dot (`.`) characters. `System.out` is an example of such a name: it identifies the specific object through which we call the `println` method. Dot-separated names appear throughout Java programs with great regularity.

White Space

White space in Java programs refers to the collective set of blank characters used to visually separate the elements of the code: spaces, tabs, and newline characters. The term itself evokes the idea that, on a white page with dark text, the gaps between words and symbols appear as empty white space. How a programmer chooses to employ white space has a meaningful impact on the visual structure of the code and how easily a reader can navigate and comprehend it.

KEY CONCEPT

Appropriate use of white space makes a program easier to read and understand.

With the exception of its role in separating adjacent tokens, white space is completely invisible to the compiler and has no effect on how a program behaves. This gives programmers considerable freedom in how they lay out their code. That said, this freedom should be used thoughtfully: lines should be broken at logical boundaries, and indentation should be applied consistently so that the structural hierarchy of the program is immediately apparent.

Because the compiler disregards white space, there is technically no single correct way to format a program. To illustrate this, consider the extremes. One approach would be to pack as many elements as possible onto each line with minimal spacing — as demonstrated by the `Lincoln2` program in Listing 1.2, which produces identical output to the original `Lincoln` program despite its dramatically compressed formatting.

```
// LISTING 1.2

//*****

// Lincoln2.java Java Foundations

//

// Demonstrates a poorly formatted, though valid, program.

//*****

public class Lincoln2 {public static void main(String[] args) {

System.out.println("A quote by Abraham Lincoln:");

System.out.println("Whatever you are, be a good one.");}}
```

OUTPUT

A quote by Abraham Lincoln:

Whatever you are, be a good one.

```
// LISTING 1.3

//*****

// Lincoln3.java Java Foundations
//
// Demonstrates another valid program that is poorly formatted.
//*****

    public      class
Lincoln3
{
    public
    static
    void
main
    (
String
        []
    args
    )
    {
        System.out.println      (
            "A quote by Abraham Lincoln:"
        )
;        System.out.println
            (
                "Whatever you are, be a good one."
            )
    }
}
```

```
)  
  
;  
}  
  
}
```

OUTPUT

A quote by Abraham Lincoln:
Whatever you are, be a good one.

KEY CONCEPT

You should adhere to a set of guidelines that establishes the way you format and document your programs.

At the opposite extreme, each token could be placed on its own line with wildly inconsistent indentation, as illustrated by `Lincoln3` in Listing 1.3.

All three versions of `Lincoln` are syntactically legal and will produce the same output when executed. Yet from a human perspective, they could hardly be more different. The latter two examples are studies in poor style — they make the code needlessly difficult to read and interpret. In professional settings, developers are typically expected to conform to a defined style guide. Software companies routinely enforce coding standards that all team members must follow. Regardless of the context, it is wise to select a consistent set of formatting practices and apply them habitually, with the goal of making your code as clear and approachable as possible for anyone who reads it.

1.2 Program Development

Getting a program up and running requires working through a series of interconnected steps. The program must first be authored in a suitable programming language such as Java. It then needs to be converted into a form the computer can actually execute. Problems can surface at any point along the way and must be identified and resolved. A variety of software tools exist to assist throughout the entire development process. Let's take a closer look at each of these aspects.

Programming Language Levels

Imagine someone giving directions to a friend. Those directions could be communicated in any number of languages — English, Russian, Italian — but the underlying route remains the same regardless of which language is used. What changes is the manner of expression. The friend, of course, can only follow the directions if they understand the language being spoken.

The same principle applies to programming. A given problem can be solved by writing a program in any of a wide range of languages — Java, Ada, C, C++, C#, Pascal, Smalltalk — and the fundamental purpose of the program remains unchanged regardless of the choice. What varies is the specific syntax used to express the instructions and how those instructions are organized. The computer, like the friend receiving directions, can only act on instructions it is capable of understanding.

Programming languages fall into four broad categories, which roughly trace the historical arc of their development:

- machine language
- assembly language
- high-level languages
- fourth-generation languages

For a program to actually run on a computer, it must ultimately be expressed in that machine's native language — its machine language. Each CPU architecture defines its own machine language, which is why a program compiled specifically for a Sun Workstation's Sparc processor cannot run directly on a Dell PC with an Intel processor.

Every machine language instruction accomplishes only a single, elementary operation. Adding two numbers together, for instance, might require four or more separate instructions. Yet a processor can execute millions of such instructions per second, meaning that enormous complexity can be achieved through the rapid chaining of many simple steps.

KEY CONCEPT

All programs must be translated to a particular CPU's machine language in order to be executed.

Machine language is represented entirely as strings of binary digits, making it extraordinarily difficult for humans to read or write. In the earliest days of computing, programs had to be entered through switches or other equally laborious mechanisms — a process that was both time-consuming and highly error-prone.

These challenges spurred the development of assembly language, which substituted binary patterns with mnemonics: short, English-like abbreviations representing commands or data values. Working with words rather than raw binary is considerably more manageable for programmers. However, assembly code cannot be run directly on a processor; it must first be converted into machine language. Each assembly instruction maps to a single corresponding machine instruction, which means the improvement in expressiveness is

modest — the programmer still works at a very low level of abstraction. Both assembly language and machine language are classified as low-level languages.

KEY CONCEPT

High-level languages allow a programmer to ignore the underlying details of machine language.

The dominant mode of software development today relies on high-level languages. These languages use constructs that closely resemble human-readable English, making them far more accessible to write and understand. A single statement in a high-level language can accomplish what might take many hundreds of machine-level instructions to perform. The term "high-level" reflects the fact that such languages operate at a level of abstraction far removed from the raw machine instructions that actually execute. Java is a high-level language, as are Ada, C++, Smalltalk, and numerous others.

Figure 1.2 offers a side-by-side comparison of the same addition operation expressed in a high-level language, assembly language, and machine language — all targeting a Sparc processor.

The high-level expression is clean, intuitive, and immediately readable — essentially indistinguishable from ordinary algebra. The assembly version is moderately readable, but more verbose and less natural. The machine language representation is practically indecipherable by human eyes, and the snippet shown in Figure 1.2 is only a fragment: the full binary encoding of that single addition operation exceeds 400 bits.

High-level languages free developers from having to think about the machine language of the underlying processor. The trade-off is that high-level code must be translated into machine language before it can be executed.

Some languages operate at yet another level of abstraction beyond conventional high-level languages. These may offer built-in facilities for generating reports automatically or interacting with databases. Such languages are known as fourth-generation languages — or 4GLs — so named because they followed the first three historical generations of programming: machine, assembly, and high-level.

Editors, Compilers, and Interpreters

The process of building new software relies on a collection of specialized programs called software tools. Three of the most fundamental are the editor, the compiler, and the interpreter.

The editor is the starting point: it is the environment in which you write your program and save it to a file. Many editors exist, each offering different features and capabilities. Becoming fluent with your editor of choice pays real dividends, since it directly affects how efficiently you can write and revise code.

Figure 1.3 illustrates the high-level flow of the program development cycle. After writing and saving the program, the next step is to translate the high-level source code into a form the

computer can execute. If the translation reveals errors, you return to the editor to correct them. Once the translation completes successfully, you run the program and assess its output. If the output falls short of expectations, or if you want to extend the program's capabilities, you cycle back to the editor once more.

The translation of source code into machine-executable form can be accomplished in several different ways. A compiler is a program that takes code written in one language and produces equivalent code in another. The input is referred to as source code, and the target output is expressed in the destination language. In many traditional compilers, source code is translated directly into machine language for a specific processor. Once this translation is done, the resulting executable can be launched on demand without repeating the compilation step.

KEY CONCEPT

A Java compiler translates Java source code into Java bytecode, a low-level, architecture-neutral representation of the program.

An interpreter operates along similar lines but with a key distinction: rather than translating the entire program up front, an interpreter interlaces translation and execution. It processes one small unit of the source code — typically a single statement — translates it, executes it, then moves on to the next. This eliminates the need for a separate compilation phase, but the program generally runs more slowly because translation happens anew during each execution.

Java employs a hybrid approach that draws on both compilation and interpretation, as depicted in Figure 1.4. The Java compiler converts source code into Java bytecode — a compact, low-level representation that resembles machine language but is not tied to any specific hardware. The Java interpreter then reads and executes this bytecode on the target machine. Alternatively, a bytecode compiler can translate the bytecode into native machine language for more efficient execution on a given platform.

The key advantage of bytecode is its hardware independence. Because it is not bound to any particular processor architecture, Java programs are inherently portable — the same bytecode runs on any machine that has a compatible Java interpreter or bytecode compiler. The only prerequisite is that such an interpreter or compiler exists for the target platform.

Since Java source code is first compiled into the lower-level bytecode representation, interpretation is considerably more efficient than interpreting high-level source code directly. Running a program from bytecode is still somewhat slower than executing native machine code, but for the vast majority of applications the performance difference is negligible. And when maximum efficiency is required, bytecode can always be compiled all the way down to machine code.

Development Environments

A software development environment is the complete set of tools a developer uses to create, test, and refine programs. Some environments are freely available, while others with more

advanced capabilities carry a cost. Environments that consolidate multiple tools into a single unified application are known as integrated development environments, or IDEs.

Any development environment will include certain foundational tools — at minimum, a Java compiler and interpreter. Many also bundle a debugger, which assists in tracking down and eliminating errors. Additional tools that may be present include documentation generators, archiving utilities, and visual aids for exploring program structure.

Sun Microsystems, the originator of Java, offers the Java Software Development Kit (SDK) — also known as the Java Development Kit (JDK) — as a free download for multiple hardware platforms from java.sun.com. A copy is also included on the CD accompanying this book.

The SDK is not an integrated environment. Its compilation and execution tools are invoked from the command line and offer no graphical user interface — no windows, menus, or buttons. Nor does it include a built-in editor, though any plain-text editor can fill that role.

KEY CONCEPT

Many different development environments exist to help you create and modify Java programs.

Sun also provides a full-featured Java IDE called NetBeans (www.netbeans.org), which wraps the SDK's capabilities in a cohesive, GUI-driven interface. IBM offers a comparable environment called Eclipse (www.eclipse.org). Both are open source projects, developed collaboratively by large communities of contributors and available at no cost.

A research team at Auburn University has produced jGRASP, another free Java IDE included on this book's accompanying CD and downloadable from www.jgrasp.org. Beyond the standard development tools, jGRASP provides graphical visualizations of program structure that can aid in comprehension and learning.

Numerous other Java development environments are available as well, and several are bundled on the book's CD. Selecting the right environment and investing time in learning its features thoroughly can significantly accelerate your productivity as a developer.

Syntax and Semantics

Every programming language is governed by its own unique syntax. Syntax rules specify precisely how the elements of the language — its keywords, symbols, and identifiers — may be arranged and combined to form valid statements. These rules are non-negotiable; a program that violates them cannot be processed. Several Java syntax rules have already come up: the requirement that an identifier cannot begin with a digit is one example; the use of braces to delimit classes and methods is another. A formal definition of Java's syntax rules can be found in Appendix J, and specific rules are called out throughout the text as they arise.

The compiler enforces all syntax rules during translation. If any statement in the program violates these rules, the compiler reports the violation and refuses to produce bytecode. Java's syntax bears a strong resemblance to that of C and C++, which means developers with a background in either of those languages will find much of Java's structure immediately familiar.

The semantics of a programming statement define what that statement actually does when executed. Programming languages are designed to be unambiguous, meaning each statement has exactly one valid interpretation — no more, no less. This stands in sharp contrast to natural languages like English or Italian, which are riddled with ambiguity. The same sentence can often support two or more equally valid readings. Consider:

Time flies like an arrow.

Most readers will interpret this as a metaphor: time passes quickly, just as an arrow flies through the air. But if we treat "time" as a verb and "flies" as a noun, the sentence could describe the act of measuring the speed of a particular insect. And if we push the interpretation even further, it could be read as a statement of preference by a peculiar insect called a "time fly" — much like saying fruit flies like a banana.

The point is that a single English sentence can legitimately carry multiple meanings. A programming language cannot tolerate this kind of ambiguity. If a given instruction could be interpreted in two different ways, a computer would have no principled basis for choosing one over the other.

KEY CONCEPT

Syntax rules dictate the form of a program. Semantics dictate the meaning of the program statements.

Errors

Software, particularly during the development phase, is susceptible to various categories of problems. The term "computer error" is frequently misapplied in everyday language and means different things in different contexts. From an end user's perspective, any unexpected outcome while working with software might be labeled a computer error. Consider a scenario where a \$23 purchase is incorrectly billed as \$230 on a credit card statement. When the error is corrected, the card company might attribute it to a "computer error." But computers don't introduce random distortions into data on their own. They execute the instructions they are given and operate on the data they are provided. If the instructions are flawed or the data is wrong, the results will be wrong too. This is the essence of the well-known programming adage: "garbage in, garbage out."

KEY CONCEPT

The programmer is responsible for the accuracy and reliability of a program.

As a developer, you will encounter three distinct categories of errors:

- compile-time error
- run-time error
- logical error

The compiler validates the syntax of your code as it processes it. Any statement that violates the language's syntactic rules triggers a syntax error. The compiler also performs type-checking and can identify cases where operations are attempted on incompatible data types, even when the syntax itself is technically legal. Any error caught during this phase is called a compile-time error. When a compile-time error occurs, the compilation process fails and no bytecode is produced.

KEY CONCEPT

A Java program must be syntactically correct or the compiler will not produce bytecode.

The second category arises not during compilation but during execution. A run-time error causes the program to terminate unexpectedly. A classic example is an attempt to divide by zero — an undefined mathematical operation that causes the program to crash and halt immediately. The most resilient programs are those described as robust: they anticipate potential failure conditions and handle them gracefully rather than allowing the program to crash. In Java, many run-time problems manifest as exceptions, which can be caught and managed through structured error-handling mechanisms.

The third and often most elusive category is the logical error. A program afflicted by a logical error compiles cleanly and runs without crashing, but its output or behavior is incorrect. This might manifest as a miscalculated value, a graphical element appearing in the wrong position, or a sequence of operations occurring in the wrong order. Identifying logical errors requires careful testing — running the program with known inputs and comparing the actual results against what was expected. When a discrepancy is found, the developer must trace it back to its origin in the code and make the appropriate correction. This process of locating and eliminating defects is called debugging. Logical errors can be particularly challenging to diagnose because their visible symptoms may be far removed from their underlying cause in the code.

1.3 Problem Solving

Software creation encompasses far more than the act of writing code. While knowing how to edit and run a program is certainly necessary, the true essence of software development lies in problem solving. Every program we write exists to address a specific problem. In general terms, problem solving unfolds across several distinct steps:

1. Understanding the problem.
2. Designing a solution.
3. Considering alternatives to the solution and refining the solution.
4. Implementing the solution.
5. Testing the solution and fixing any problems that exist.

Although this framework applies broadly to problem solving in many domains, it proves especially effective in the context of software development. These steps do not follow a rigid

linear sequence — many of the activities will overlap and run concurrently. Nevertheless, at some point during development, every one of these steps deserves careful and deliberate attention.

The first step — understanding the problem — may seem self-evident, yet neglecting it has been the root cause of countless failed or misdirected software efforts. When we attempt to solve a problem we do not fully grasp, we frequently end up solving the wrong problem entirely, or at minimum pursuing directions that lead us astray.

Every problem comes with its own problem domain: the real-world context and concepts that are central to finding a valid solution. If we are tasked with writing a program to score a bowling match, for instance, the problem domain encompasses the complete rules of the game. To arrive at a sound solution, we must develop a thorough and accurate understanding of that domain.

KEY CONCEPT

Problem solving involves breaking a solution down into manageable pieces.

The foundation of good solution design is decomposition — breaking the problem into pieces small enough to be tackled individually. Virtually no meaningful problem can be solved through a single monolithic task. Instead, a solution is built from a collection of smaller, cooperating tasks that collaborate to fulfill a larger purpose. In software development, this means we do not write one enormous, undifferentiated program. We design discrete components, each responsible for a specific part of the overall solution, and then assemble them into a coherent whole.

Our initial instinct toward a solution is not always the best one. We must remain open to alternatives and be willing to refine our approach as our understanding deepens. The sooner we entertain alternative designs, the less costly it is to change course.

Implementing the solution is the step in which the design is translated into a concrete, working form. In software development, this is the phase where actual code gets written. A common misconception is that programming is primarily about writing code. In reality, the design phase — the act of thinking through and structuring the solution — is typically the more intellectually demanding and creative of the two. Translating a well-considered design into code in a specific language is, by comparison, more mechanical.

Throughout the development process, testing plays an indispensable role. We test our solution at multiple stages to surface errors so that they can be corrected. Testing cannot offer a guarantee that no undiscovered problems remain, but it steadily builds our confidence that the solution is sound and viable.

Throughout this text, we explore techniques for designing and implementing programs with elegance and clarity. While we will frequently dive into the technical details, we must never lose sight of the overarching goal: solving problems.

1.4 Software Development Activities

Since the purpose of software development is fundamentally to solve problems, it should come as no surprise that the activities comprising the development process closely mirror the general problem-solving steps outlined in the previous section. Specifically, any rigorous software development effort involves four core activities:

- Establishing the requirements.
- Creating a design.
- Implementing the design.
- Testing.

KEY CONCEPT

The effort put into design is both crucial and cost-effective.

One might wish that these four activities could be carried out in strict sequence, one after another, in a clean and predictable progression. In practice, however, they rarely work that way. They overlap, feed back into one another, and interact in ways that are difficult to anticipate. Let's take a brief look at each one.

Software requirements define what a program is supposed to accomplish. They capture the tasks a program must perform without prescribing how those tasks are to be carried out. Requirements are frequently documented in a formal artifact known as a functional specification. At their core, requirements are a precise articulation of the problem to be solved. Until we have a clear and accurate picture of what we are trying to build, we cannot build it effectively.

In an academic setting, students are usually handed their requirements in the form of a problem assignment. Even then, those requirements may need to be discussed and interpreted carefully. In a professional context, the client — the person or organization commissioning the software — typically provides an initial set of requirements. These initial requirements, however, are often incomplete, ambiguous, or internally inconsistent. The developer must engage with the client in an iterative dialogue to refine and resolve all outstanding questions about what the system is expected to do.

Requirements frequently address user interface considerations, including output formatting, screen layouts, and graphical components. More broadly, they define the characteristics that make the software genuinely useful to its intended users. They may also impose constraints on the system, such as performance thresholds that must be met.

A software design describes how a program will fulfill its requirements. The design identifies the classes and objects the program will need, specifies how they interact with one another, and defines the relationships between classes. At a lower level of detail, design also addresses how individual methods will carry out their responsibilities.

No civil engineer would construct a bridge without a detailed design. The same discipline applies to software. A significant proportion of software failures can be traced directly to insufficient design effort. Experience has demonstrated repeatedly that investing time and

thought into the design phase pays for itself many times over, reducing both the time and cost required to deliver a working solution.

During the design phase, alternatives must be actively sought out and evaluated. The initial design is rarely the optimal one, and course corrections are far easier to make at this stage than after implementation has begun.

Implementation is the process of writing the source code that will bring the design to life. More precisely, it is the act of translating a design into a specific programming language. Many developers make the mistake of treating implementation as the primary activity of software development. In reality, it should be the least creative phase — the important decisions are made when establishing requirements and crafting the design. Implementation is, in large part, the mechanical realization of choices already made.

Testing is the process of verifying that a program correctly solves the intended problem across all the conditions and constraints it must operate within. It involves running the program repeatedly with a range of inputs and scrutinizing the results with care. But testing extends well beyond executing the finished program. In some form, testing should be woven into every stage of development. The requirements, for example, should be validated by reviewing them directly with the client to confirm they accurately reflect the intended problem.

1.5 Object-Oriented Programming

As noted earlier in this chapter, Java is an object-oriented language — meaning that objects serve as the central entities around which programs are built. The central thesis of this book is that good software is constructed by defining objects that collaborate and communicate with one another.

The principles of object-oriented software development have a long history, stretching back roughly as far as high-level programming languages themselves. The language Simula, created in the 1960s, already embodied many of the characteristics that define the modern object-oriented approach. During the 1980s and 1990s, object-oriented programming rose to prominence — driven in large part by the emergence of languages like C++ and Java — and it has since become the dominant paradigm in commercial software development.

One of the most compelling qualities of the object-oriented approach is how naturally objects map to real-world entities. A software object can represent an employee within a company, for instance. We would create one object per employee, each encapsulating the behaviors and characteristics relevant to our needs. In this way, object-oriented programming allows us to model the real situations our programs are meant to represent — making it inherently more intuitive to reason about solutions, which is ultimately the entire point of writing a program.

Let's explore the specific qualities of the object-oriented approach that make problem solving more tractable.

Object-Oriented Software Principles

A thorough understanding of object-oriented programming rests on a solid grasp of the following foundational concepts:

- object
 - attribute
 - method
 - class
 - encapsulation
 - inheritance
 - polymorphism

Beyond these core terms, there is a rich body of associated ideas that enable solutions to be tailored in countless ways. This book is designed to develop your understanding of these concepts progressively and organically. The discussion that follows offers a high-level overview of each idea, establishing the vocabulary and painting the broader picture before we dive into the details.

KEY CONCEPT

Each object has a state, defined by its attributes, and a set of behaviors, defined by its methods.

We established earlier that an object is a fundamental building block of a program. A software object frequently corresponds to something concrete in the problem domain — a bank account, for example. Every object possesses two essential characteristics: a state and a set of behaviors. State refers to the object's current condition — the values that define what it is at any given moment. For a bank account, part of that state is the current balance. Behaviors are the actions the object is capable of performing. For a bank account, typical behaviors include the ability to accept deposits and process withdrawals.

Alongside objects, a Java program also works with primitive data — raw fundamental values such as integers and characters. Objects tend to represent entities that are more complex or conceptually rich than primitive values alone.

An object's attributes are the individual pieces of data it holds internally. These may be primitive values or references to other objects. A bank account object, for instance, might store a floating-point number representing the account balance, along with a string holding the account holder's name. Taken together, the current values of all an object's attributes constitute its state.

As introduced earlier, a method is a named grouping of programming statements. When a method is called, those statements execute. Each object is associated with a set of methods, and it is those methods that define the object's potential behaviors. To give a bank account

the ability to accept deposits, we write a method containing the statements necessary to update the balance accordingly.

Every object is defined by a class. A class is the template or blueprint from which objects are created. Think of the blueprints an architect produces when designing a house. The blueprints specify the house's essential features — its room layout, window placements, door positions, electrical outlets, and so on. Once those blueprints exist, any number of houses can be constructed from them, as illustrated in Figure 1.5.

In certain respects, the houses built from the same blueprint are distinct from one another. They occupy different locations, carry different addresses, contain different furnishings, and are home to different people. Yet in equally important ways, they are fundamentally the same house — sharing the same floor plan and all the structural characteristics defined by the blueprint. To produce a meaningfully different house, you would need a different blueprint.

A class is the blueprint of an object. It specifies what data objects of that type will hold and defines the methods that govern their behavior. Critically, however, a class is not itself an object — just as a blueprint is not a house. In general, a class does not allocate storage for data. Each object created from a class has its own dedicated storage, which is what allows each object to maintain its own independent state.

KEY CONCEPT

A class is a blueprint of an object. Multiple objects can be created from one class definition.

Once a class has been defined, any number of objects can be instantiated from it. After defining a class to represent the concept of a bank account, for example, we can create as many individual bank account objects as we need, each tracking its own distinct balance independently of the others.

An object should be encapsulated, meaning it takes responsibility for protecting and managing its own internal data. An object should be self-governing in the sense that its state should only be modified through its own methods. The design goal is to prevent other objects from reaching directly into an object and altering its state without going through the prescribed channels.

New classes can be built on top of existing ones through a mechanism called inheritance. Inheritance allows the definition of one class to be derived from another already-defined class, effectively reusing and extending the capabilities of the parent. This is a powerful form of software reuse that capitalizes on the natural similarities between related types of classes. One class can serve as the foundation for multiple derived classes, and those derived classes can in turn give rise to further subclasses, forming a hierarchical structure. Attributes and methods defined in a parent class are inherited by its children, which pass them along to their own children in turn.

As a concrete example, we might build a hierarchy of classes representing various types of financial accounts. Characteristics shared across all account types would be defined in a high-level base class, while the distinguishing features of each specific account type would be captured in the corresponding derived class.

Polymorphism is the principle that allows us to interact with different but related types of objects in a uniform and consistent manner over time. It enables the construction of powerful, flexible solutions to problems that involve working with collections of varied but conceptually related objects.

Several of these core object-oriented concepts are illustrated in Figure 1.6. A full understanding of these ideas is not expected at this stage — the vast majority of this book is devoted to developing that understanding in depth. This overview is intended simply to lay the groundwork and orient you to the landscape ahead.

Summary of Key Concepts

- A computer system consists of hardware and software that work in concert to help us solve problems.
- This book focuses on the principles of object-oriented programming.
- Comments do not affect a program's processing; instead, they serve to facilitate human comprehension.
- Inline documentation should provide insight into your code. It should not be ambiguous or belabor the obvious.
- Java is case sensitive. The uppercase and lowercase versions of a letter are distinct.
- Identifier names should be descriptive and readable.
- Appropriate use of white space makes a program easier to read and understand.
- You should adhere to a set of guidelines that establishes the way you format and document your programs.
- All programs must be translated to a particular CPU's machine language in order to be executed.
- High-level languages allow a programmer to ignore the underlying details of machine language.
- A Java compiler translates Java source code into Java bytecode, a low-level, architecture-neutral representation of the program.
- Many different development environments exist to help you create and modify Java programs.
- Syntax rules dictate the form of a program. Semantics dictate the meaning of the program statements.
- The programmer is responsible for the accuracy and reliability of a program.
- A Java program must be syntactically correct or the compiler will not produce bytecode.
- Problem solving involves breaking a solution down into manageable pieces.
- The effort put into design is both crucial and cost-effective.
- Each object has a state, defined by its attributes, and a set of behaviors, defined by its methods.
- A class is a blueprint of an object. Multiple objects can be created from one class definition.